Khalani Whitepaper

Wojciech Klaudiusz Zaborowski
Wojciech Kołowski
Mateusz Pyzik

31 October 2025

1 The main part of the litepaper comes here

Appendices

A A deeper dive into Axi

Axi (pronounced "Axee") is the smart contract language of Khalani. It is a purely functional programming language with a strong, static, linear type system and support for formal reasoning about programs in classical higher-order logic. Among mainstream programming languages, it is most similar to Haskell (in spirit) and OCaml (syntactically). It was also heavily inspired by proof assistants based on Type Theory, like Coq, Lean and Agda.

In this appendix we will have a brief, not too technical look at the philosophy and language features of Axi, together with a few simple code examples.

A.1 Philosophy

Axi's architecture is unique in that the language is split into two strictly separate layers: the programming layer and the logic layer.

The Programming Layer of Axi is a purely functional language based on System F with algebraic data types and a resource-aware type system. It aims to make programming pleasant and comprehending smart contract code easy, while providing essential guarantees to all on-chain code:

- **Type safety**. Well-typed programs can't go wrong, i.e. they can't crash (unless they run out of gas), can't perform nonsensical operations like adding a number and a string, and can't result in any other runtime type error.
- **Memory safety**. Well-typed programs can't result in memory errors, like writing to a random memory location, nor can they have memory leaks.
- **Resource safety**. Well-typed programs can't access resources they don't have permission to, can't delete tokens that are not supposed to be deletable and can't create money out of thin air.

The Logic Layer of Axi is a language for reasoning about Axi programs in classical higher-order logic. It allows expressing arbitrarily complex mathematical properties, thus giving programmers the ability to precisely specify the business logic and behaviour of their smart contracts. These specifications can then be either tested with random inputs before deployment, checked for concrete inputs at runtime, or proved for all possible inputs using logic. The last of these, proving, provides guarantees much more powerful than those of the type system alone, while being as credible and trustworthy as the type system. The use of the logic layer is entirely optional, i.e. programmers are never forced to engage with any part of it, although competition will encourage programmers to do so in the hopes of outcompeting other programmers' smart contracts by providing stronger guarantees.

Strict separation. Logic can reason about programs, but programs cannot manipulate logical entities, like theorems or proofs, at runtime. This one-way interaction keeps the language relatively simple, while still making it possible to express almost all functional programming idioms and all possible mathematical properties and their proofs.

Guarantees all the way down. We not only want Axi to provide strong guarantees about its programs, but we also want to provide strong guarantees about Axi itself. We plan to use the Coq proof assistant to formally verify significant portions of Axi's metatheory, including the aforementioned properties of type and resource safety, as well as the consistency of Axi's logic. We are also considering formal verification of the reference implementation of Axi. The goal of these efforts is to ensure that the guarantees that Axi provides are trustworthy and will stay so forever, as long as the consensus protocol is followed.

A.2 Language features

In this section, we will take a closer look at Axi's language features.

A.2.1 The Programming Layer

Axi is, at its core, a purely functional programming language descending from the ML family. Thus, it supports all the usual features of such languages:

- **Purity amd immutability**. In Axi, all values are pure and immutable – they cannot be null, cannot throw exceptions and once created, they cannot be changed. This facilitates reasoning about and comprehending code, thus rendering Axi smart contracts transparent and auditable.

- **First-class, higher-order and anonymous functions**. Functions are first class citizens. They can be passed as arguments to other functions, returned from other functions as results and created on the fly. Axi encourages a programming style in which data is processed by pipelines built from basic building blocks like mapping, reducing, filtering and so on.

- **Algebraic data types and exhaustive pattern-matching**. Instead of clunky boolean flags and cascades of ifs, Axi makes it easy to model the problem domain with disjoint unions and define functions on them by pattern matching and explicitly handling all possible cases.

- **Strong static type system**. Axi's type system makes it easy to work with high-level abstractions, like merkle trees or state machines, instead of low-level primitives, like integers or byte arrays. It is lightweight, not placing too many demands on the programmer, but still catches the most egregious errors – it can easily protect programmers from confusing with each other such objects as an integer, a string, an address, a transaction hash, a token amount and a cryptographic secret.

- **Polymorphism and interfaces**. Axi's type system features polymorphism (also known as "generics") as well as interfaces inspired by Haskell's type classes. These features make it easy to implement data structures and functions that work for all types that support a certain behaviour, for example a hash map that works for all types of ERC-20-compliant tokens bridged from an external blockchain.

Example 1. *An example program*

```
data type List (A : Type) : Type where
  nil : List A
  cons : A -> List A -> List A

map @(A : Type) @(B : Type) (f : A -> B)
: List A -> List B
| nil => nil
| cons x xs => cons x (map f xs)

mapRevApp @(A : Type) @(B : Type) (f : A -> B)
: List A -> List B -> List B
| nil, ys => ys
| cons x xs, ys => mapRevApp f xs (cons (f x) ys)

revApp @(A : Type)
: List A -> List A -> List A
| nil, ys => ys
| cons x xs, ys => revApp xs (cons x ys)
```

However, these are only the standard features that plenty of functional programming languages have. The killer feature, as far as the programming layer is concerned, are linear types, which allow the type system to enforce correct resource usage discipline. This aspect of Axi's type system was inspired by linear logic and its cousin, Quantitative Type Theory, and is an improvement upon an analogous feature of the type system of Move, the programming language of the Sui and Aptos blockchains.

In an Axi program, every variable is assigned its *quantity*, which can be either: zero, one, at most one, at least one, unrestricted. The variable's quantity determines how many times it can be used in the program, with too many or too few uses resulting in a type error. For example, the typical variable holding an NFT will have a quantity of one, forcing the program to do something with the NFT before it terminates: maybe send the NFT to the transaction's originator, maybe trade it for something else and handle that something else properly according to its quantity, or maybe send it to somebody as a gift. What the program cannot do in such a situation is forget the NFT and do nothing with it, effectively making it "lost" in the guts of the chain, nor copy the NFT and have two instead of just the original one.

To make programmers' life easier, every type is also assigned its "default" quantity. Every variable will be assigned this quantity unless the programmer decides otherwise. An important piece of terminology: a type whose default quantity is unrestricted is called a *data type*, whereas a type whose default quantity is either one, at least one or at most one, is called a *resource type*. There are no types whose default quantity is zero. Elements of data types are called data, whereas elements of resource types are called resources.

Example 2. *Conservation of money*

```
coinForgery (x : KhalaniCoin) : KhalaniCoin = coinMerge x x
//           ^                                           ^ ^
//           |                                           | |
//           \-----------------------------------\       | |
//                                               |       | |
// type error: a duplicate use of a resource 'x' -/       | |
// of type 'KhalaniCoin : Type1'                           | |
// first use -------------------------------------------/ |
// second use ----------------------------------------------/
// hint: remove the duplicate use
// or declare the type to be in 'Type+' or 'Type' to allow it
coinBlackhole (x : KhalaniCoin) : Unit = ()
//             ^                         ^
//             |                         |
//             \--------------------\    |
//                                  |    |
// type error: unused resource 'x' -/    |
// of type 'KhalaniCoin : Type1'         |
// hint: use the resource in body -------/
// or declare the type to be in 'Type?' or 'Type' to allow it
```

A.2.2 The Logic Layer

Axi's logic layer consists of typed classical higher-order logic with equality whose domains of discourse are the types from the programming layer. This means the logic consists of:

- **Propositional connectives and constants**. Standard propositional connectives ($\Longrightarrow$, $\wedge$, $\vee$, $\Longleftrightarrow$, $\sim$) and constants ($\top$, $\bot$) can be used to build up complex propositions from simpler ones.
- **Quantification over terms and types**. Axi supports universal and existential quantification. We can quantify over terms of a given type ($\forall x : A, \ldots$) as well as over types themselves ($\forall A : \texttt{Type}, \ldots$), no matter whether we're dealing with data, resources or programs.
- **Higher-order quantification**. We can also quantify over logical entities like propositions ($\forall P : \texttt{Prop}, \ldots$), predicates ($\forall P : A \to \texttt{Prop}, \ldots$), relations ($\forall R : A \to A \to \texttt{Prop}, \ldots$), logical connectives ($\forall C : \texttt{Prop} \to \texttt{Prop} \to \texttt{Prop}, \ldots$) or even quantifiers themselves ($\forall Q : (A \to \texttt{Prop}) \to \texttt{Prop}, \ldots$). This ability of logic to talk not only about the underlying programming language, but also about itself, gives it great expressive power and allows it to make use of all of mathematics when reasoning about the behaviour of programs.
- **Equality**. The built-in equality relation ($=$) makes it possible to precisely reason about programs' input-output behaviour.

Proving is carried out in a proof language based on the Curry-Howard correspondence: proofs of implications and universal quantifiers are like functions, proofs of existential quantifiers and conjunctions are like pairs, proofs of disjunctions are like disjoint union constructors, etc. Negation and biconditional are handled as in intuitionistic logic - instead of being primitives, negation ($\sim P$) is a shorthand for an implication into falsehood ($P \implies \bot$), whereas biconditional ($P \iff Q$) is a shorthand for a conjunction of two implications ($(P \implies Q) \wedge (Q \implies P)$). The principle of *proof by contradiction*, which allows to prove $((\sim P \implies \bot) \implies P)$ for any proposition P, makes the logic classical and thus intimately familiar to all mathematicians.

One novel feature that distinguishes our proof language is that the proofs can be presented in two styles: either in a *proof term style*, similar to how proofs look in Agda and Idris (as well as Coq's and Lean's manually written proof terms), or in a *tactic style*, similar to how proofs usually look in Coq or Lean. The difference between Axi and languages that also support both styles, like Coq and Lean, is that in these languages each style is its own separate proof language, whereas in Axi there is a single proof language which, thanks to a clever parser, can be formatted either like terms or like tactics. This feature will make it easier to accomodate computer scientists and mathematicians familiar with either of the aforementioned languages.

Equality is handled as in Martin-Löf Type Theory - proved by reflexivity and eliminated with the so-called J rule, which is a principle that allows to substitute equals for equals in the remaining part of the proof. Because proofs are hard to write using these primitives, Axi provides a convenient feature called *chaining*, which allows reasoning with long chains of equations by writing down and justifying each step separately, similarly to how equality is usually handled with pen and paper. Besides equality, this feature also supports implications, biconditionals and other transitive relations.

Example 3. *A simple proof*

```
theorem revApp-map @(A : Type) @(B : Type) (f : A -> B) (xs : List A) :
  forall @(ys : List A), revApp (map f xs) ys === mapRevApp f xs ys
proof
  induction xs with
  | nil =>
    assume ys
    chaining
      === revApp (map f nil) ys
      === revApp nil ys
      === ys
      === mapRevApp f nil ys
  | cons x (xs & ind xsIH) =>
    assume ys
    chaining
      === revApp (map f (cons x xs)) ys
      === revApp (cons (f x) (map f xs)) ys
      === revApp (map f xs) (cons (f x) ys)
      === mapRevApp f xs (cons (f x) ys)    by xsIH (cons (f x) ys)
      === mapRevApp f (cons x xs) ys
qed
```

A.2.3 Rich interfaces

Last but not least, the killer feature of the logic layer are *rich interfaces*. These are the same interfaces from the programming layer that we already mentioned. However, what we did not mention yet is that in addition to declarations of constants and functions, they may also contain specifications, i.e. properties expected to be true of the declared constants and functions. When implementing such an interface, besides defining the constants and functions, we must also provide proofs of the specifications.

Rich interfaces let us precisely define what is expected of various pieces of the blockchain infrastructure. For example, we could define an interface of a DEX, with functions like placing orders, canceling orders and collecting tokens from filled orders, and specify these using properties like "all orders that are not filled can be canceled", "the user can recover his money after canceling an order", etc.

Any smart contract that attempts or pretends to implement a DEX would become much more credible and trustworthy if it implemented this DEX interface. This is because the interface is a relatively short and readable way of making promises and guarantees regarding the service's behaviour – instead of having to read plenty of code, which moreover might be difficult to understand, the user only needs to search the code for a phrase like "implements the DEX interface" and read a few short logical specifications. Even though the latter task is still outside of most people's capabilities, it is vastly simpler than having to do your own audit of the code.

Moreover, this task can be offloaded to *social learning*, i.e. after some time it might become common knowledge, a good practice, etc., that any purported DEX should implement the DEX interface. In such a scenario, the due diligence is reduced to just looking for the phrase "implements the DEX interface" in source code, which is in most people's reach.

Example 4. *Coin interface*

```
interface Coin (A : Type1) where
  coinAmount : A -> Nat * A
  coinZero : A
  coinMerge : A -> A -> A
  coinSplit : CoinAmount -> A -> Option A * A
  theorem coinZero-spec :
    let x = coinZero in
    let (a, _) = coinAmount x in
    a === 0
  theorem coinMerge-spec :
    forall @(x : A) @(y : A),
    let (a, _) = coinAmount x in
    let (b, _) = coinAmount y in
    let z = coinMerge x y in
    let (c, _) = coinAmount z in
    c === a + b
  theorem coinSplit-spec :
    forall @(n : Nat) @(coin : A),
    let (a, _) = coinAmount coin in
    match coinSplit n coin with
    | (none, whole) =>
      (n <= a) === false /\
      let (b, _) = coinAmount whole in
      b === a
```

```
    | (some exact, change) =>
      (n <= m) === true /\
      let (b, _) = coinAmount y in
      b === n /\
      let (c, _) = coinAmount z in
      c === a - n

record type KhalaniCoin : Type1 where
  amount : Nat

instance Coin KhalaniCoin where
  coinAmount (coin : KhalaniCoin) : Nat * KhalaniCoin =
    (coin.amount, coin)
  coinZero = record where amount = 0
  coinMerge (coin1 coin2 : KhalaniCoin) : KhalaniCoin =
    record where amount = coin1.amount + coin2.amount
  coinSplit (amount : Nat) (coin : KhalaniCoin) :
      Option KhalaniCoin * KhalaniCoin =
    if amount <= coin.amount then
      ( some (record where amount = amount)
      , record where amount = coin.amount - amount
      )
    else
      (none, coin)
  theorem coinZero-spec :
    let x = coinZero in
    let (a, _) = coinAmount x in
    a === 0
  proof
    refl
  qed
  // other proofs elided
```

B Interacting with the ledger in Axi

In this appendix, we will see how Axi programs can interact with Khalani's ledger. First we will discuss a simplified ledger model, which we will use in the remaining code examples, and look at the primitive (buil-in) types and functions that represent it in Axi. Then we will give an overview of the big idea behind Axi's interaction with the ledger, which we might call the "resource lifecycle". Finally we will discuss a slightly longer example of an on-chain service, together with example transactions that set this service up and use it.

B.1 A simplified ledger model and Axi primitives

We will use the word *ledger* to refer to the state of the blockchain. Things on the ledger have their addresses. The three kinds of addresses are *accounts*, *package addresses* and *cell addresses* (or just addresses for short).

Accounts are addresses that act as the identities of external actors. They allow them to interact with the blockchain, similarly to EOAs (Externally Owned Accounts) on Ethereum.

All Axi code deployed onto the ledger is stored in a big codebase, divided into *packages*. Package addresses are means of locating these packages in this codebase. Packages are further subdivided into smaller units, called *modules*, which hold definitions of types, interfaces, constants and functions.

Data and resources on the ledger are stored in *cells*, also called *objects*. Each cell has an address and a type, and stores exactly one value of that type. Cells also have some metadata which won't concern us for now.

Objects are either *owned* or *shared*. Owned objects have exactly one owner who is an account. Shared objects do not have an owner. Shared objects are further subdivided into *mutable shared objects*, which contain resources and can be interacted with by anybody (any account or object) according to the interface set by the package defining the object's type, and *immutable shared objects*, which contain data but not resources and can be read by anybody, but not modified.

The state of the ledger is modified by transactions. Every transaction has a *transaction originator* and a *transaction body*, which is an Axi function that describes how to modify the ledger. Transactions also have some additional metadata which won't concern us here.

Example 5. *Axi primitives for interacting with the ledger*

```
primitive
  // provides access to ledger and transaction metadata
  Ledger : Type1

  // a unique address of a cell or an account
  Address : Type

  // an address that signed the currently executed transaction
  whoami : Ledger -> Address * Ledger

  // create a new cell owned by a specified address
  transfer : forall (A : Type1),
    Address -> A -> Ledger -> Address * Ledger

  // destroy an existing cell owned by transaction signer
  receive : forall @(A : Type1), Address -> Ledger -> Option A * Ledger

  // create a new mutable shared cell, accessible by everyone
  share : forall (A : Type1), A -> Ledger -> Address * Ledger

  // the cell type, parameterized by the type of its contents
  Cell : Type1 -> Type 1

  // replace and retrieve the contents of a cell
  exchange : A -> Cell A -> A * Cell A

  // get an owned or a shared cell and set its new version
  update : forall @(A : Type1),
    Address -> Ledger -> Option (Cell A * (Cell A -> Ledger -> Ledger)) *
    Ledger

  // create a new immutable shared cell, accessible by everyone
  freeze : forall (A : Type), A -> Ledger -> Address * Ledger

  // read contents of an immutable cell
  peek : forall @(A : Type), Address -> Ledger -> Option A * Ledger
```

The above code listing shows the most important primitives (built-ins) available in Axi. `Address` is the data type of addresses, including accounts. `Ledger` is an abstract type that represents (the current state of) the ledger. The body of a transaction is a function of type `Ledger -> Ledger`.

Every function that interacts with the ledger takes (the current version of) the ledger as input and also returns (the new version of) the ledger as output. In general, the type of such a function is `A -> Ledger -> B * Ledger`, i.e. it takes a value of some type `A` and the ledger, and returns a value of some type `B` and the (updated) ledger.

The transaction body will, in general, be composed of many such functions. It will get the initial input of type `Ledger` from the transaction context and then pass it to the first of these functions, which will in turn pass it to the second one, and so on. Because `Ledger` is a resource type, the type system guarantees that values of this type cannot be copied or deleted. This means that the ledger is acted upon and then passed further and further until the transaction body returns the final modified version of it.

An example function that needs the ledger is `whoami`, which consumes the old ledger and returns the account address that is the originator of the current transaction, together with a new ledger.

`transfer` is a function that takes an account address and a value of some type, together with the ledger, creates a new cell on the ledger that contains this value and belongs to that account, and returns the address of the created cell together with the new ledger.

`receive` is a function that takes a cell address, together with the ledger, and returns an optional value of some type `A`, together with the new ledger. If the transaction originator is not the owner of the cell, or the value stored in the cell is not of type `A`, then the operation fails and the result is `none`. Otherwise the operation succeeds and the result is `some a`, where `a` is the value that was stored in this cell.

`share` is a function that takes a value of some type `A`, together with the ledger, creates a new mutable shared cell that contains this value, and returns the address of the created cell together wtih the new ledger.

`freeze` is a function that takes a value (which must be data and not a resource), together with the ledger, creates a new immutable shared cell that contains this value, and returns the address of the created cell together with the new ledger.

`peek` is a function that takes an address, together with the ledger, reads the contants of an immutable shared cell under this address, and returns an optional value of some type `A` together with the new ledger. If the address does not point to an immutable shared cell, or the value stored in that cell is not of type `A`, then the operation fails and the result is `none`. Otherwise the operation succeeds and the result is `some a`, where `a` is the data stored in the cell.

`update` is a function that takes a cell address, together with the ledger, and returns the new ledger, together with the result of type `Option (A * (A -> Unit))`. If the cell is owned but the transaction originator is not the owner, or the address does not point to a shared mutable cell, or the value stored in the cell is not of type `A`, the operation fails and the result is `none`. Otherwise the operation succeeds and the result is `some (a, f)`. Here `a` is the value stored in the cell, whereas `f` is a callback which, when given `a'` as input, sets `a'` as the new contents of the cell and returns a dummy output. Note that `A -> Unit` is a resource type, so the type system makes sure that the callback is eventually called.

B.2 The resource lifecycle

A new resource type is created by a transaction that deploys a package of Axi code that contains the definition of this resource type. The only exception is Gas, the native coin used to pay transaction fees, which is built-in, so it already exists at the origin block. Because packages live on-chain forever and cannot be deleted, a resource type, once it comes to life, will also exist forever.

The life cycle of a particular resource, i.e. an element of some resource type, consists of three stages:

- **Creation**. At some point the resource is created according to the rules set by the package that contains the definition of the resource's type. After creation, the resource is placed on the ledger in a cell owned by somebody.
- **Transaction(s)**. The resource might participate in transactions, if its owner wishes so. The owner might transfer his ownership of the resource freely, whereas programmatic interaction of the resource with other objects happens only according to the rules set by the defining package.
- **Destruction (optional)**. If the defining package permits it, the resource might at some point be destroyed. Examples of destructible resources are stablecoins that are backed by an off-chain reserve, where a coin gets destroyed when its off-chain collateral is removed from the treasury, and a wrapper for an asset bridged from another blockchain, which gets destroyed when the external asset ceases to be bridged.

The most important part of the lifecycle, in which most resources will spend almost all their life, is the phase when they can participate in transactions. An ordinary transaction (i.e. one which does not deploy a new package) consists of an arbitrary Axi program that transforms the state of the ledger, which we'll call the *transaction body*, and of some metadata, most importantly the *transaction originator*. This program, when executed, interacts with the ledger in three steps:

- **Withdraw**. First, resources are "withdrawn" from the ledger into the program. The ledger's ownership model ensures the safety of this step, for example by making sure that the program cannot withdraw resources that don't belong to the transaction originator.
- **Transform**. Second, the program transforms its resources by performing various computations, exchanging the resources for other resources and moving them around, etc. The safety of this step is guaranteed by the type system. The resources cannot get unlawfully destroyed or copied because the type system keeps an eye on them, nonsensical operations cannot be performed on them, because the program is well-typed, they cannot get lost in the jungle of pattern matching, because exhaustiveness is checked, and so on.
- **Deposit**. Third, the program "deposits" the transformed resources back onto the ledger. The safety of this step is partially ensured by the type system, which checks that all resources held by the program are deposited back. However, since resources can be freely transferred, it's up to the transaction author to make sure that they are transferred to the correct address and not to somebody else.

B.3 A longer example

Example 6. *Donut Shop*

```
// Donut - simple purchasable object
record type Donut : Type1 where
  serialNumber : Nat

// Donut shop - shared object
record type DonutShop : Type1 where
  owner : Address
  price : Nat
  balance : KhalaniCoin
  nextSerialNumber : Nat

// Create a donut shop
init (ledger : Ledger) : Address * Ledger =
  let (sender, ledger) = whoami ledger in
  let shop : DonutShop = record where
    owner = sender
    price = 1000
    balance = coinZero
    nextSerialNumber = 0 in
  share shop ledger

// One can buy a donut for a set price
buyDonut (payment : KhalaniCoin) (shop : DonutShop) :
    Option Donut * KhalaniCoin * DonutShop =
  let (splitResult, change) = coinSplit shop.price payment in
  match splitResult with
  | none => (none, change, shop)
  | some paid =>
    let newBalance = coinMerge shop.balance paid in
    let newSerialNumber = shop.nextSerialNumber in
    let newShop : DonutShop =
      record shop where
        balance = newBalance
        nextSerialNumber = newSerialNumber + 1 in
    let donut : Donut = record where serialNumber = newSerialNumber in
    (some donut, change, newShop)

// Consume donut and get nothing useful in return
eatDonut (donut : Donut) : Unit =
  let record {} = donut in ()

// Collects profits from shop (succeeds only for owner)
collectProfits (shop : DonutShop) (ledger : Ledger) :
    KhalaniCoin * DonutShop * Ledger =
  let (sender, ledger) = whoami ledger in
  if shop.owner == sender then
    let profits = shop.balance in
    let newShop = record shop with balance = coinZero in
    (profits, newShop, ledger)
  else
    (coinZero, shop, ledger)
```

C The core calculus of Axi

In this section we will give a draft of the formal definition of the core calculus which lies at the foundation of Axi. This draft is not complete, as it covers

only the core of the programming layer of Axi – functions, polymorphism, some primitive types and the machinery behind linear typing – but is missing some more advanced programming features (like type operators), the more mundane features like algebraic data types, modules and packages, as well as the logic layer.

Click to see slides with the theory